Machine Learning Methods for Finance

Unsupervised Learning - Generative Models

Prof. Mike Aguilar
Economics Department and Fuqua School of Business, Duke University

https:/www.linkedin.com/in/mike-aguilar-econ

Outline

1. Generative Modeling
   - Generating Returns
   - Classical
   - Diffusion Models
   - GAN

Motivation

- These notes accompany the Duke Economics course *Machine Learning for Finance*.
- The course focuses on how machine learning methods can be used to design, evaluate, and implement systematic investment strategies.
- Our goal is to introduce a range of ML techniques with enough mathematical structure to build intuition, while avoiding unnecessary formalism.
- Throughout the notes we use a simple return–momentum signal as a running example. Although momentum is not always the ideal illustration for every technique, using a single, familiar signal provides a consistent thread. This is especially true when we transition to real data and empirical implementation.
- The methods we discuss are broadly applicable across many signals, asset classes, and investment settings.

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

- **Clustering**
  Group similar assets by momentum characteristics
- **Dimension Reduction**
  Condense multiple momentum measures into fewer factors
- **Generative Modeling**
  Simulate returns to derive momentum structure

Supervised Learning

- **Classification**
  Predict positive/negative returns using momentum
- **Regression**
  Predict cross-sectional returns using momentum inputs

Time Series Modeling

- **Unsupervised**
  Group assets by temporal behavior, then measure momentum
- **Supervised**
  Forecast returns based on persistence

Outline

1. Generative Modeling
   - Generating Returns
   - Classical
   - Diffusion Models
   - GAN

Outline

1. Generative Modeling
   - Generating Returns
   - Classical
   - Diffusion Models
   - GAN

Intro to Generative Models

Generative aka Simulation

- Generative Models are a flexible form of unsupervised (or semi-supervised) learning and can be deployed in many applications within finance
- In the following we will focus on perhaps the most common, and certainly the most relevant to our use case
- Namely, we will simulate asset returns
- Note that this exercise is cast in a time series context; i.e. we will simulate the evolution of returns over time
- However, the simulation methods here are i) not built to capture temporal persistence (see our Time Series notes) and ii) can be applied in myriad cross sectional settings.

Modeling Returns

Q: Why usually simulate returns, not prices?

- Most approaches that simulate (i.e. Generate) asset behavior tend to focus on returns, not prices
- Prices tend to be non stationary
  - Stationary: no trend& similar mean and variance within small windows of time
- Most of our estimation techniques rely upon stationary process, and so we tend to avoid modeling prices directly.
- On the other hand, returns tend to be stationary, and thus are the typical object of interest

Defining Returns

Q: Simple vs Log? Definition and Use Case?

- Simple Returns are computed as $R_t = P_t/P_{t-1} - 1$
  - Best for performance reporting
- Log Returns are computed as $r_t = ln(P_t/P_{t-1}) = ln(1 + R_t)$
  - Best for regression, inference, simulation

Q: Net vs Gross? Definition and Use Case

A:

- The returns listed above (R and r) are NET
- Gross Simple Returns $= 1 + R$
- Gross Log Returns $= 1 + r$
- Use Gross Returns (either type) for multi period return calculations.
  e.g. $R_{Month} = \prod_{t=1}^{4}(1 + R_{week}) - 1$

Simulating with Simple Returns is Difficult

Q: Problems with simulating Gaussian Simple Returns?

Two problems arise if we assume $R_t \sim iid\ N(\mu, \sigma^2)$

1. Limited Liability ($R_t \geq -1$) can be violated, which means simulated prices could be negative
2. Even though R_t is Gaussian, multi-period simple returns $R_t(H) = \prod_{j=1}^{H-1}(1 + R_{t-j}) - 1$ are not

Solution:

- Assume Gross Simple Returns are log Normal
- $r_t = ln(1 + R_t) \sim N(\mu, \sigma^2)$

Resolving Limited Liability with Log Returns

$$r_t = \ln(1 + R_t)$$

$$\exp(r_t) = \exp\big(\ln(1 + R_t)\big) = 1 + R_t, \quad \text{range}(\exp) = (0, \infty)$$

$$\Rightarrow \quad 1 + R_t > 0 \implies R_t > -1$$

$$\Rightarrow \quad \frac{P_t}{P_{t-1}} - 1 > -1$$

$$\Rightarrow \quad \frac{P_t}{P_{t-1}} > 0 \implies P_t > 0 \quad (\text{if } P_{t-1} > 0).$$

Resolving Lack of Multi-Period Gaussianity with r

For a horizon of length H, define the multi-period gross return:

$$1 + R_t(H) \;=\; \prod_{j=0}^{H-1} \big(1 + R_{t-j}\big).$$

Using $1 + R_{t-j} = \exp(r_{t-j})$, we obtain

$$1 + R_t(H) = \prod_{j=0}^{H-1} \exp\left(r_{t-j}\right) = \exp\Big(\sum_{j=0}^{H-1} r_{t-j}\Big).$$

Therefore, the multi-period log return is

$$r_t(H) = \ln\left(1 + R_t(H)\right) = \sum_{j=0}^{H-1} r_{t-j}.$$

Note: The sum of Gaussian random variables is Gaussian. Hence, if each r_{t-j} is Gaussian (under appropriate independence or dependence conditions), then $r_t(H)$ is Gaussian.

Building a Momentum Signal via Simulation

Q: How might we use simulation to build a momentum based signal?

1. Compute the most recent momentum value for asset A
2. Simulate many returns paths for asset A
3. Compute momentum for each simulated return path
4. Estimate the mean and standard deviation of simulated momentum
5. Compute the z-score of the observed momentum relative to the simulated distribution
6. Repeat for all assets in the universe

$$s_{i,t} = \mathbf{1}\left[p^{(+)}(X_{i,t}, Z_t)\right] - \mathbf{1}\left[p^{(-)}(X_{i,t}, Z_t)\right]$$
$$p^{(+)} = 1[z_i^{Mom} > 2], \quad p^{(-)} = 1[z_i^{Mom} < -2]$$

- $X_{i,t}$: momentum on asset i
- Z_t: momentum for all other assets

Simulation vs Historical

Q: What are the advantages of simulation over historical estimates?

A:

- We could have computed the mean and standard deviation of OBSERVED returns/momentum and used these to construct the z-score.
- Instead, we used the mean and standard deviation of SIMULATED returns/momentum.
- Advantages of simulation:
  - **Forward-looking assumptions:** We can model what we think will happen (e.g., changing regimes or structural breaks).
  - **Scenario analysis:** We can explore what could happen, including scenarios not observed historically.
  - **Distributional analysis:** We can examine the range and likelihood of possible outcomes, including tail events.
  - **Complex outcomes:** We can simulate return paths to study quantities such as momentum, cumulative returns, and drawdowns.
- **Caution:** Simulation does not eliminate estimation uncertainty. Results depend on assumptions used to generate simulated data.

Outline

1. Generative Modeling
   - Generating Returns
   - Classical
   - Diffusion Models
   - GAN

Parametric Simulation

- Classical parametric simulation assumes a Data Generating Process (DGP) for returns
- A DGP typically has two components:

$$r_t = \underbrace{\mu_t}_{\text{systematic component}} + \underbrace{\epsilon_t}_{\text{innovation (shock)}}$$

- Simplest case (pure noise):

$$r_t = \epsilon_t, \qquad \epsilon_t \sim N(0, \sigma^2)$$

- More structured example:

$$r_t = \alpha + \beta r_{t-1} + \epsilon_t$$

Building a Momentum Signal with Gaussian Simulation

1. Compute the most recent momentum value for asset A
2. **Estimate the mean and standard deviation of asset A's returns**
3. Simulate many return paths for asset A **using a parametric model based on the mean and standard deviation**
4. Compute momentum for each simulated return path
5. Estimate the mean and standard deviation of simulated momentum
6. Compute the z-score of the observed momentum relative to the simulated distribution
7. Repeat for all assets in the universe

$$s_{i,t} = \mathbf{1}\Big[p^{(+)}(X_{i,t}, Z_t)\Big] - \mathbf{1}\Big[p^{(-)}(X_{i,t}, Z_t)\Big]$$
$$p^{(+)} = 1[z_i^{Mom} > 2], \quad p^{(-)} = 1[z_i^{Mom} < -2]$$

- $X_{i,t}$: momentum on asset i
- Z_t: momentum for all other assets

Kernel Density Estimation (KDE)

KDE is a nonparametric approach to simulation

- Parametric simulation: assume a specific distribution (e.g., Gaussian), estimate a few parameters, and simulate from that assumed shape.
- KDE simulation: make no global distributional assumption; instead, approximate the density as a sum of small local densities centered at the observed data, then simulate from that empirical shape.
- The Kernel density function is $\frac{1}{nh}\sum_{i=1}^{n} K(\frac{x-x_i}{h})$, where n is the number of observations, $h > 0$ is the smoothing parameter (aka bandwidth), and $K(.)$ is the kernel function, which integrates to 1.
- Many Kernels are possible, such as Gaussian, Uniform, Triangular, etc...

From Data to a Density

How KDE builds a smooth analogue of a histogram

1. Form empirical distribution

If we treat the raw data as a density:

$$\hat{f}_{\text{emp}}(x) = \frac{1}{n} \sum_{i=1}^{n} \delta(x - x_i).$$

- Each observation is a *point mass* aka "spike"
- Graphically: vertical spikes at the observed values
- This is the raw input into KDE

From Data to a Density cont'd

2. KDE: replace spikes with kernels Recall that a histogram would smooth the "spikes" into "bins".
Instead of bins, the KDE:

- Replaces each spike $\delta(x - x_i)$ with a small smooth "bump"
- The bump is the *kernel*, centered at x_i

$$\delta(x - x_i) \longrightarrow \frac{1}{h} K\left(\frac{x - x_i}{h}\right)$$

- Each observation contributes a local density "hill"
- Shape determined by the kernel K
- Width controlled by the bandwidth h

From Data to a Density cont'd

Stacking local density "bumps"

3. Build the density curve by stacking bumps

The KDE density curve is simply:

$$\hat{f}(x) = \frac{1}{n} \sum_{i=1}^{n} (\text{kernel bump centered at } x_i).$$

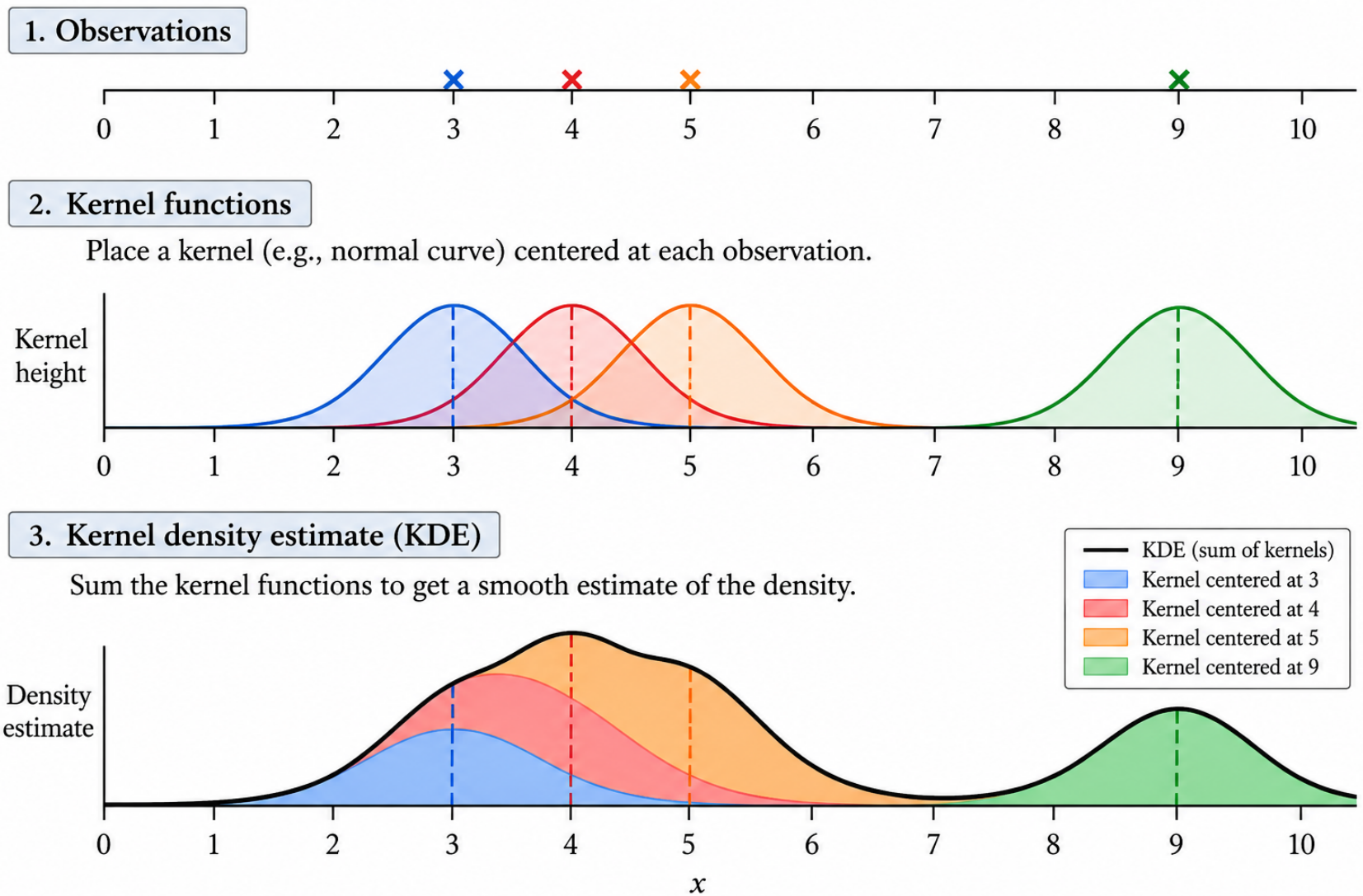
Graphically:

- Place one kernel at each data point
- Scale each kernel so its total area equals $1/n$
- Add all kernels together

The resulting density curve is:

- Smooth (if the kernel is smooth)
- Continuous (no bins)
- Higher in regions where many kernel bumps overlap

KDE Visualization



Simulating from a Kernel Density Estimate

Q: How simulate data from the KDE?

1. Randomly select an observed data point x_i (each with probability $1/n$)
2. Draw noise ε from the kernel, scaled by the bandwidth
   - Kernel controls the shape, while Bandwidth controls the scale
   - A wider bandwidth stretches the kernel so that noise draws are larger
   - For consistency we usually use the same bandwidth for simulation as we use for estimation
3. Form a simulated draw:

$$\tilde{x} = x_i + \varepsilon$$

Building a Momentum Signal with KDE Simulation

Q: How use KDE to form a momentum-based signal?

1. Compute the most recent momentum value for asset A
2. Simulate many return paths for asset A **using KDE for a given Kernel and bandwidth**
3. Compute momentum for each simulated return path
4. Estimate the mean and standard deviation of simulated momentum
5. Compute the z-score of the observed momentum relative to the simulated distribution
6. Repeat for all assets in the universe

$$s_{i,t} = \mathbf{1}\Big[p^{(+)}(X_{i,t}, Z_t)\Big] - \mathbf{1}\Big[p^{(-)}(X_{i,t}, Z_t)\Big]$$
$$p^{(+)} = 1[z_i^{Mom} > 2], \quad p^{(-)} = 1[z_i^{Mom} < -2]$$

- $X_{i,t}$: momentum on asset i
- Z_t: momentum for all other assets

Outline

1. Generative Modeling
   - Generating Returns
   - Classical
   - Diffusion Models
   - GAN

How we'll use Diffusion Models

- Like parametric and KDE, we will use Diffusion models to simulate returns
- Diffusion models are inherently non parametric, and in that sense are similar to KDE
- As in both parametric and KDE methods, Diffusion models train and then simulate. However, Diffusions are trained by simulating noisy versions of the data and learning to *reverse* that process.

How Diffusion Models Work

Diffusion models generate realistic samples by reversing a gradual noise (diffusion) process

- Start with real data x_0; e.g. a block of returns for asset i; $x_0 = \{r_{t-13}, r_{t-12}, ...\}$
  - As we have previously warned, we are using these generative models to generate realistic blocks of data. We will attempt to capture true temporal dynamics in our Time Series discussions.
- Gradually corrupt the data by adding small amounts of noise
- After many steps, the data becomes indistinguishable from pure noise
- Train a neural network to reverse this corruption, step by step
- Generate new samples by starting with pure noise and using the trained neural network to progressively remove noise, step by step.

Diffusion as Two Markov Processes

Forward process (fixed, no learning):

$$x_0 \xrightarrow{q(x_1|x_0)} x_1 \xrightarrow{q(x_2|x_1)} \cdots \xrightarrow{q(x_L|x_{L-1})} x_L \approx \mathcal{N}(0, I)$$

- Gradually adds Gaussian noise
- Eventually destroys the structure in the data
- The reseacher specifies a noise schedule $\{\beta_l\}_{l=1}^{L}$ that controls the noise added at each step

Reverse process (learned):

$$x_L \xrightarrow{p_\theta(x_{L-1}|x_L)} x_{L-1} \xrightarrow{\cdots} x_1 \xrightarrow{p_\theta(x_0|x_1)} x_0$$

- Learns to reverse the noising process
- Parameterized by a neural network

Forward Diffusion Process

The forward process has a closed-form expression:

$$x_l = \sqrt{\bar{\alpha}_l}\, x_0 + \sqrt{1 - \bar{\alpha}_l}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I)$$

where

$$\bar{\alpha}_l = \prod_{s=1}^{l} (1 - \beta_s)$$

- $\bar{\alpha}_l$ measures cumulative signal retention
- β_l controls the noise added at step l
- As l increases, the signal fades and noise dominates
- For sufficiently large l, x_l is approximately standard Gaussian

This formulation allows sampling x_l directly from x_0 without simulating all intermediate steps.

Training Diffusion Models

Rather than learning $p_\theta(x_{l-1} \mid x_l)$ directly, DDPMs (Denoising Diffusion Probabilistic Model) train the model to predict the noise added at each step.

Objective (score matching / noise prediction):

$$\min_\theta \ \mathbb{E}_{l,\, x_0,\, \epsilon}\left[\left\|\epsilon - \epsilon_\theta(x_l, l)\right\|^2\right]$$

where:

$$x_l = \sqrt{\alpha_l}\, x_0 + \sqrt{1-\alpha_l}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).$$

- $\epsilon_\theta(x_l, l)$ is a neural network
- The network learns to estimate the noise conditional on the corrupted data and noise level
- The noise-prediction loss is derived from a simplified version of the variational training objective

Noise Prediction

Q: We said that DDPMs train the model to "predict the noise added at each step". How can the neural network predict noise if noise is random?

A: The network cannot predict random noise in advance. Instead, it learns to estimate the noise after observing the corrupted data. It exploits the statistical structure of real data to distinguish likely signal from noise.

Generating New Samples with Diffusion

Sampling procedure:

- Start from pure noise: $x_L \sim \mathcal{N}(0, I)$
- Iteratively apply the learned reverse transitions:

$$x_{l-1} \sim p_\theta(x_{l-1} \mid x_l)$$

- Continue until reaching x_0

Each step slightly denoises the sample; many small corrections produce a realistic final draw.

What Does DDPM Mean?

DDPM = Denoising Diffusion Probabilistic Model

- **Denoising**: the model learns to remove noise
- **Diffusion**: noise is added gradually over many steps
- **Probabilistic**: each step defines a conditional distribution.

Diffusion Models: Construction vs Simulation

Model Training (Learning)

Goal: Learn the reverse process that undoes noise.

Inputs

- Real data samples $x_0 \sim p_{\text{data}}$
- Known forward noising process $q(x_l \mid x_{l-1})$

What happens

- Add noise to data (forward diffusion)
- Train a neural network to predict / remove noise
- Optimize parameters θ via a loss function

Output

- A trained diffusion model $p_\theta(x_{l-1} \mid x_l)$

Simulation (Sampling)

Goal: Generate synthetic data.

Inputs

- Pure noise $x_L \sim \mathcal{N}(0, I)$
- Trained model parameters θ

What happens

- Iteratively apply the learned reverse process
- Gradually denoise the sample

Output

- Simulated sample $\tilde{x}_0 \sim p_\theta$

Building a Momentum Signal with Diffusion Models

Q: How use a Diffusion model to form a momentum-based signal?

1. Compute the most recent momentum value for asset A
2. Simulate many return paths for asset A **using a trained Diffusion model, including choices of noise schedule $\{\beta_l\}_{l=1}^{L}$, reverse-model architecture, and sampling scheme**
3. Compute momentum for each simulated return path
4. Estimate the mean and standard deviation of simulated momentum
5. Compute the z-score of the observed momentum relative to the simulated distribution
6. Repeat for all assets in the universe

$$s_{i,t} = \mathbf{1}\Big[p^{(+)}(X_{i,t}, Z_t)\Big] - \mathbf{1}\Big[p^{(-)}(X_{i,t}, Z_t)\Big]$$
$$p^{(+)} = 1[z_i^{Mom} > 2], \quad p^{(-)} = 1[z_i^{Mom} < -2]$$

- $X_{i,t}$: momentum on asset i
- Z_t: momentum for all other assets

Outline

1. Generative Modeling
   - Generating Returns
   - Classical
   - Diffusion Models
   - GAN

How we'll use Generative Adversarial Networks

- Like the other generative approaches of parametric, KDE, and Diffusion models, we will use GAN's to simulate returns
- GANs are inherently non parametric, akin to KDE and Diffusion
- GANs are trained by learning a generator G that produces synthetic samples $\tilde{x}$ indistinguishable from real data x by playing an adversarial game against a discriminator D.

How GANs Work

- Start with real data x; e.g. a block of returns for asset i; $x = \{r_{t-13}, r_{t-12}, \ldots\}$
  - As we have previously warned, we are using these generative models to generate realistic blocks of data. We will attempt to capture true temporal dynamics in our Time Series lectures.
- $G_\theta(z)$ maps noise z into synthetic data; it learns the data distribution implicitly.
- $D_\phi(x)$ outputs a probability that a sample is real.
- Training is a *minimax game*: D tries to detect fakes; G tries to fool D.
- At equilibrium, G reproduces the true data distribution and D cannot distinguish real from synthetic.
- Unlike VAEs, GANs learn without specifying an explicit likelihood model.
- Common variants stabilize training (WGAN, WGAN-GP, spectral norm).

GAN Architecture

Learn a generator G that produces synthetic samples $\tilde{x}$ indistinguishable from real data x by playing an adversarial game against a discriminator D.

Architecture:

$$z \sim p(z) \xrightarrow{\text{Generator } G_\theta} \tilde{x} \qquad x,\, \tilde{x} \xrightarrow{\text{Discriminator } D_\phi} \text{prob(real)}$$

Optimization Problem (Unsupervised):

$$\min_{G_\theta} \max_{D_\phi} \Big[\mathbb{E}_{x \sim p_{\text{data}}} \big[\log D_\phi(x) \big] + \mathbb{E}_{z \sim p(z)} \big[\log(1 - D_\phi(G_\theta(z))) \big] \Big]$$

Notation:

- $x \sim p_{\text{data}}$: Average over observations drawn from the real data distribution.
- $z \sim p(z)$: Average over random noise inputs drawn from a specified distribution.

GAN Mappings

Generator / Discriminator and the Role of Embeddings

Generator Mapping:
A typical generator takes noise and optional conditioning variables:

$$h = \phi(W_z z + W_c c + b_h), \qquad \tilde{x} = G_\theta(z, c) = W_g h + b_g,$$

where c may include static features or a learned embedding identifying the asset.

Discriminator Mapping:

$$h_D = \phi(W_x x + W'_c c + b_D), \qquad D_\phi(x, c) = \sigma(W_d h_D + b_d),$$

which estimates $\Pr(\text{real} \mid x, c)$.

Building a Momentum Signal with a GAN

Q: How use a GAN to form a momentum-based signal?

1. Compute the most recent momentum value for asset A
2. Simulate many return paths for asset A **using a trained GAN with a given choice of Generator, Discriminator, and adversarial loss**
3. Compute momentum for each simulated return path
4. Estimate the mean and standard deviation of simulated momentum
5. Compute the z-score of the observed momentum relative to the simulated distribution
6. Repeat for all assets in the universe

$$s_{i,t} = \mathbf{1}\left[p^{(+)}(X_{i,t}, Z_t)\right] - \mathbf{1}\left[p^{(-)}(X_{i,t}, Z_t)\right]$$
$$p^{(+)} = 1[z_i^{Mom} > 2], \quad p^{(-)} = 1[z_i^{Mom} < -2]$$

- $X_{i,t}$: momentum on asset i
- Z_t: momentum for all other assets